# Image Fragment Reconstruction: A Siamese Network Approach via Adjacency Prediction

May 2, 2026

## Contents

# 1 Problem Statement

Let $\mathcal{I} = \{I_1, \ldots, I_N\}$ be a set of $N$ source images, each of spatial resolution $H \times W \times 3$. Each image is divided into a regular $G \times G$ grid of non-overlapping fragments of size $\frac{H}{G} \times \frac{W}{G} \times 3$, yielding $G^2$ fragments per image and $M = N \cdot G^2$ fragments in total. In the experimental setting used here $N = 10$, $H = W = 64$, $G = 4$, giving fragments of size $16 \times 16 \times 3$ and $M = 160$ fragments per batch.

Let $f_k \in \mathbb{R}^{16\times16\times3}$ denote fragment $k$, and let $s(k) \in \{1, \ldots, N\}$ be its source image. The fragments from all $N$ images are pooled into a single unordered collection. The task is:

> *Given the pooled collection of $M$ fragments with no positional or source information, assign each fragment to one of $N$ groups such that all fragments assigned to the same group originate from the same source image.*

Formally, the goal is to recover a partition $\hat{\mathcal{C}} = \{\hat{C}_1, \ldots, \hat{C}_N\}$ of $\{1, \ldots, M\}$ such that for each predicted cluster $\hat{C}_n$ there exists a source image $I_s$ with $s(k) = s$ for all $k \in \hat{C}_n$.

**Constraints.** Two constraints make this non-trivial:

1. **No positional information.** The original grid coordinates of each fragment are not provided at inference time. The model may not use absolute position as a feature.

2. **No labels during training.** The source identity $s(k)$ is never provided to the model as a supervisory signal. The model must learn from structural properties of the fragments themselves.

**Known structure.** Although labels are withheld, the following quantities are known and may be exploited:

- The total number of source images $N = 10$ (so $k = N$ clusters are sought).

- Each source image contributes exactly $G^2 = 16$ fragments, so all clusters have equal size.

- Spatial adjacency within a source image is fully determined by the grid structure and can be derived without labels: fragments at grid positions $(r, c)$ and $(r', c')$ are adjacent if and only if $|r - r'| + |c - c'| = 1$.

The third point is the key observation exploited by the proposed approach (Section 2).

# 2 Core Idea

The central observation is that two fragments cut from the same image and placed next to each other in the grid share a common boundary — colours bleed over, textures continue, and objects span tiles. This visual continuity is a strong and reliable signal that does not require any human annotation to exploit: since the experimenter creates the grid, the ground-truth adjacency of every fragment pair is known at all times.

The proposed approach uses adjacency prediction as a **pretext task** (Noroozi and Favaro 2016). The model is trained to solve a surrogate problem — predicting whether two fragments are spatially adjacent — and in doing so is forced to learn visual representations that capture source identity as a side effect. At inference time the pretext objective is discarded and the learned representations are used for clustering.

This is self-supervised throughout: no labels beyond those derivable from the grid structure are ever used.

# 3 Why Adjacency Prediction Helps

The naive contrastive approach (Section 3 of the companion document) faces a fundamental tension: intra-image fragments can be visually dissimilar (a sky tile vs. a ground tile from the same photograph), while inter-image fragments can be visually similar (two sky tiles from different photographs). Adjacency prediction addresses this directly:

- To predict whether two fragments are adjacent, the model must first determine whether they could plausibly come from the same image at all — source identity is a necessary precondition for adjacency.
- Adjacency exploits **local edge continuity** — a signal that is orthogonal to global visual similarity and therefore less susceptible to the sky-tile problem.
- The pretext task is harder than simple contrastive learning, which forces the model to learn richer representations.

# 4 Architecture: Siamese Network

The model consists of two components:

**Shared CNN encoder.** A single convolutional neural network maps each $16\times16\times3$ fragment to a 256-dimensional embedding vector. The architecture is three convolutional blocks (Conv–BatchNorm–ReLU), with max-pooling after blocks two and three, reducing the spatial dimensions from $16 \times 16$ to $4 \times 4$. The flattened $4 \times 4 \times 128$ feature map is projected to a 256-dimensional embedding via a linear layer followed by ReLU and dropout ($p = 0.3$).

The encoder is **shared** — the same weights are used for both fragments in a pair. This is the defining property of a Siamese network (Bromley et al. 1994): weight sharing ensures that the comparison is symmetric and that the encoder learns a universal fragment representation rather than one tuned to a specific input slot.

**Comparison head (MLP).** A small multi-layer perceptron takes as input a 1024-dimensional vector formed by concatenating four interaction terms between the two embeddings $\mathbf{a} = \mathrm{CNN}(f_i)$ and $\mathbf{b} = \mathrm{CNN}(f_j)$:

$$\mathbf{z}_{ij} = \big[\mathbf{a} - \mathbf{b} \parallel \mathbf{a} \odot \mathbf{b} \parallel \mathbf{a} \parallel \mathbf{b}\big] \in \mathbb{R}^{1024}$$

The difference $\mathbf{a} - \mathbf{b}$ captures asymmetric discrepancies; the element-wise product $\mathbf{a} \odot \mathbf{b}$ captures feature co-activation; and including both raw embeddings preserves their individual information. The MLP maps $\mathbf{z}_{ij}$ through a hidden layer of size 128 (ReLU) to a single scalar in $[0, 1]$:

$$p_{ij} = \sigma\big(W_2\, \mathrm{ReLU}(W_1\, \mathbf{z}_{ij} + b_1) + b_2\big)$$

# 5 Training

## 5.1 Constructing Training Labels

From a sample of 10 images cut into a 4×4 grid, each fragment has at most 4 adjacent neighbours (fewer at corners and edges). For a sample of 160 fragments the total number of pairs is:

$$\binom{160}{2} = 12{,}720$$

Each pair receives a binary label: $y_{ij} = 1$ if the two fragments are adjacent within the same image, $y_{ij} = 0$ otherwise. This label is derived entirely from the known grid positions and requires no human annotation.

## 5.2 Class Imbalance

Adjacent pairs are rare relative to non-adjacent pairs. Within one image of 16 fragments there are 24 adjacent pairs and $\binom{16}{2} - 24 = 96$ non-adjacent same-image pairs, plus all cross-image pairs. Across the full 160-fragment sample the positive rate is low. This imbalance should be addressed via weighted loss or by subsampling negatives during training.

## 5.3 Loss Function

Binary cross-entropy is the natural loss for a binary prediction task:

$$\mathcal{L} = -\frac{1}{N} \sum_{(i,j)} \Big[ y_{ij} \log p_{ij} + (1 - y_{ij}) \log(1 - p_{ij}) \Big]$$

## 5.4 Computational Cost

The CNN encoder runs once per fragment (160 forward passes per sample). All pairwise embeddings are then combined using tensor operations, and the lightweight MLP processes all 12,720 pairs. Since the expensive step (the CNN) runs only 160 times, the computational overhead is manageable.

# 6 From Pretext Task to Clustering

After training, the model produces a predicted probability $p_{ij}$ for every pair of fragments. These probabilities define a **fully connected weighted graph** over the 160 fragments, where edge weight $w_{ij} = p_{ij}$ reflects the model's confidence that fragments $i$ and $j$ are adjacent — and by extension, that they originate from the same image.

**Decision point: clustering on the graph.** **Spectral clustering** is the natural choice for a weighted graph. It uses the eigenstructure of the graph Laplacian to find clusters that are internally well-connected and externally weakly connected. With $k = 10$ known clusters, the $k$ smallest eigenvectors of the Laplacian are used.

This is more principled than applying $k$-means directly to embeddings, because the graph structure captures the full relational information between all fragment pairs rather than just the position of individual embeddings in space.

**Metric consistency.** Unlike the contrastive approach, there is no distance metric to align between training and clustering — the edge weights are probabilities produced directly by the model, and spectral clustering operates on those weights natively.

# 7 Evaluation Metric: Adjusted Rand Index

The primary evaluation metric is the **Adjusted Rand Index (ARI)**. It measures how well the predicted clusters match the true groupings by counting fragment pairs:

- A pair is a **true positive** if both fragments are in the same true group and the same predicted cluster.
- A pair is a **true negative** if they are in different true groups and different predicted clusters.
- All other pairs are errors.

The "adjusted" part corrects for chance — a random clustering would score ARI = 0, a perfect clustering scores ARI = 1, and a clustering worse than random can score below zero.

ARI is preferred over simpler metrics such as purity because purity only looks at the dominant class in each cluster and ignores how errors are distributed. For example, a cluster containing 60% of fragments from image A and 40% from image B scores the same purity as a cluster containing 60% from image A, 20% from image B, and 20% from image C — despite the latter being more disordered. ARI captures this distinction.

**NMI (Normalised Mutual Information)** is reported alongside ARI as a complementary metric. It measures how much information the predicted clusters share with the true groupings, normalised to $[0, 1]$.

# 8 Decision Points Summary

| **Decision** | **Choice** | **Alternatives** |
|---|---|---|
| Training objective | Adjacency prediction (pretext task) | Contrastive loss |
| Architecture | Siamese CNN + MLP head | Single encoder |
| Embedding combination | Diff + product + both embeddings | Concatenation only |
| Loss function | Binary cross-entropy | Focal loss (for imbalance) |
| Class imbalance | Weighted loss or negative subsampling | None |
| Clustering algorithm | Spectral clustering | $k$-means on embeddings |
| Number of clusters | $k = 10$ (known) | Discovered automatically |
| Evaluation metric | ARI, NMI | Purity |

## 9 Limitations

- Adjacency is a local signal — it says nothing directly about fragments that are far apart within the same image. The model must generalise from local adjacency to global source identity.
- The graph has 12,720 edges. For larger grids or larger samples this grows quadratically and may become a bottleneck.
- The pretext metric and the task metric are misaligned. We train binary adjacency (only $\sim$1.9% positives) but evaluate clustering. The model can achieve a useful similarity matrix (ARI $\approx$ 0.78) while only mediocre on adjacency F1 ($\approx$ 0.26), because adjacency is a sufficient but not necessary condition for same-source identity.

## 10 Further Suggestions

**Multi-task loss combining adjacency and same-image prediction.** The current loss optimises only adjacency, which is sparse ($\sim$1.9% positive pairs) and only indirectly aligned with the clustering task. A natural extension is to train two heads jointly:

$$\mathcal{L} = \lambda_{\text{adj}} \cdot \text{BCE}_{\text{adjacency}} + \lambda_{\text{same}} \cdot \text{BCE}_{\text{same-image}}$$

The same-image label $y_{ij}^{\text{same}} = \mathbb{1}[s(i) = s(j)]$ is also free (known from the grid construction), denser ($\sim$10% positives), and directly matched to what the spectral clustering will use downstream. Adjacency provides the finer-grained, harder-to-learn signal; same-image prediction provides the dense task-aligned signal. Tuning $\lambda_{\text{adj}}$ and $\lambda_{\text{same}}$ would let us trade off between the two.

**Recall-tilted adjacency objective.** At inference time, spectral clustering does not need precise per-edge probabilities — it needs the overall connectivity structure of the graph to be correct. A few false-positive edges between same-image fragments are tolerable; missed adjacent edges weaken the connectivity within a true cluster. This argues for tilting the adjacency loss toward recall, e.g. via a higher positive-class weight, focal loss with $\alpha > 0.5$, or an asymmetric BCE in which false negatives are penalised more heavily than false positives. The current weighted BCE already partially does this; further increasing the positive weight is worth ablating.

**Balanced spectral clustering.** Each source image always contributes exactly $G^2 = 16$ fragments, so the true clusters are perfectly balanced. Sklearn's `SpectralClustering` ignores this and can produce clusters of size 5 and 30. Two practical ways to enforce balance:

1. **Spectral embedding + balanced $k$-means.** Compute the spectral embedding as usual ($k$ leading eigenvectors of the normalised Laplacian), then cluster the embedding with a balanced $k$-means variant (e.g. `k-means-constrained` on PyPI, which solves a min-cost flow problem to enforce equal cluster sizes). This is a one-line replacement for the final clustering step.

2. **Optimal-transport assignment.** Compute the spectral embedding, choose 10 cluster centroids (e.g. via $k$-means), then solve a balanced assignment problem (Sinkhorn or Hungarian on a $160 \times 10$ cost matrix with capacity 16 per cluster) to assign each fragment to a cluster while respecting capacity.

Approach (1) is the lower-effort option and likely sufficient.

# TODO

- **Limit model capacity** — a small CNN has less capacity to memorise than a large one. Justify the chosen architecture size explicitly in the report.
- **Average ARI / F1 over many validation batches.** The current evaluation uses a single fresh batch of 10 images per eval step, producing high variance in the reported metrics. This affects (i) the readability of training curves, (ii) early-stopping decisions, and (iii) the saved checkpoint, which is selected on best single-batch ARI. Averaging over 20–30 fresh validation batches per eval step would reduce all three issues at the cost of a small constant factor in eval time. Expose the number of validation batches as a config parameter.

**Already implemented.** Dropout ($p = 0.3$ after the encoder's projection layer) and weight decay ($\lambda = 10^{-4}$ in the Adam optimiser) are already active in the current implementation.

# References

Bromley, Jane et al. (1994). "Signature Verification Using a Siamese Time Delay Neural Network". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 6, pp. 737–744.

Noroozi, Mehdi and Paolo Favaro (2016). "Unsupervised Learning of Visual Representations by Solving Jigsaw Puzzles". In: *European Conference on Computer Vision (ECCV)*. Springer, pp. 69–84.